EPRLens AI — ML to AWS Engineer Guide

How to run the two EPRLens models on **AWS Lambda**, with files in **Amazon S3** and results in **Amazon DynamoDB**. No SageMaker, ECR, Textract, Bedrock or API Gateway is needed.

SYNTHETIC — EPRLens Demo. Models were trained on synthetic data. Outputs support review; they are not legal findings.

What has the ML team built?

We have trained **two models**.

1. Forecast model — takes company/product packaging history and predicts **next-month packaging kg**. It is a xgboost model saved as **model.onnx**.

2. Anomaly model — takes transaction/invoice numbers and identifies unusual records. It is used together with simple deterministic **rules**. A record is flagged if any rule fires **or** the model score is above the saved threshold.

1. What EPRLens is

EPRLens AI helps Indian businesses check their packaging and EPR records *before* formal submission: it validates records, reconciles invoices, flags mistakes, forecasts packaging volume and runs what-if scenarios. It is not a CPCB portal and not a compliance guarantee.

2. What the ML team has built

Model	Input	Output	Main file
Forecast (forecast-v1)	Product history: month, packaging_kg (+ units, g/unit)	forecast_kg for next month	model.onnx
Anomaly (anomaly-v1)	One invoice with lines, or one recycling record	is_anomaly, score, rule findings	isolation_forest.joblib + rules.py

3. Files you are receiving

```
ml/artifacts/forecast/v1/
    feature_schema.json             0.002 MB
    features.py                     0.006 MB
    model.onnx                      7.101 MB
    model_metadata.json             0.004 MB
    preprocessor.joblib             0.003 MB
    requirements-lambda.txt         0.000 MB
    sample_input.json               0.002 MB
    sample_output.json              0.000 MB
    SHA256SUMS.txt                  0.001 MB
ml/artifacts/anomaly/v1/
    feature_schema.json             0.001 MB
    features.py                     0.005 MB
    isolation_forest.joblib         1.006 MB
    model_metadata.json             0.004 MB
    preprocessor.joblib             0.020 MB
    requirements-lambda.txt         0.000 MB
    rules.py                        0.009 MB
    rules_config.json               0.002 MB
    sample_input.json               0.002 MB
    sample_output.json              0.000 MB
    SHA256SUMS.txt                  0.001 MB
```

```
    threshold.json                  0.000 MB
ml/aws_examples/
    lambda_forecast_handler.py
    lambda_anomaly_handler.py
    test_forecast_local.py
    test_anomaly_local.py
```

Every folder has **SHA256SUMS.txt**. Check it after copying.

4. Forecast model

Item	What it is
Input	JSON: company_id, product_id, industry, history (oldest first; last entry = latest month)
Output	JSON: forecast_kg, target_month, model_version, model_name, status
Model file	model.onnx (runs with onnxruntime)
Preprocessor	preprocessor.joblib — feature order, industry list, seasonality table
Feature code	features.py — builds the features from the history. Do not rewrite it.
Feature schema	feature_schema.json — 30 features in a fixed order

5. Anomaly model

Item	What it is
Input	One invoice record (header, lines, linked internal record, master data, context) or one recycling record
Output	is_anomaly, anomaly_score (null for recycling), threshold, ml_flag, rule_findings
Model	isolation_forest.joblib (scikit-learn Isolation Forest)
Threshold	threshold.json — 0.5387, chosen on validation data
Rules	rules.py + rules_config.json — duplicate, missing field, quantity/unit mismatch, master-data checks, recycling checks
Feature code	features.py — builds the 19 numeric features

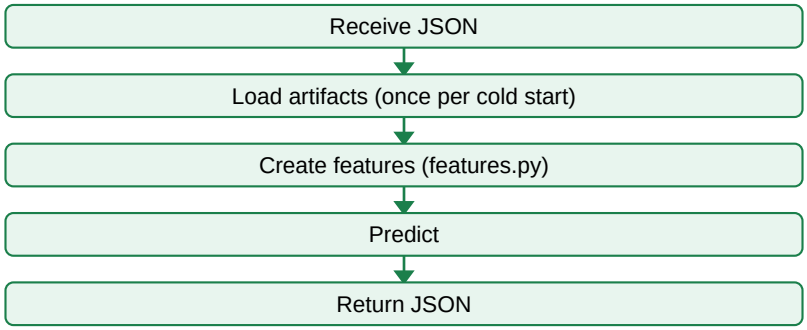
6. Where to put files in S3

Local folder	S3 location
ml/artifacts/forecast/v1/	s3://<model-bucket>/models/forecast/v1/
ml/artifacts/anomaly/v1/	s3://<model-bucket>/models/anomaly/v1/

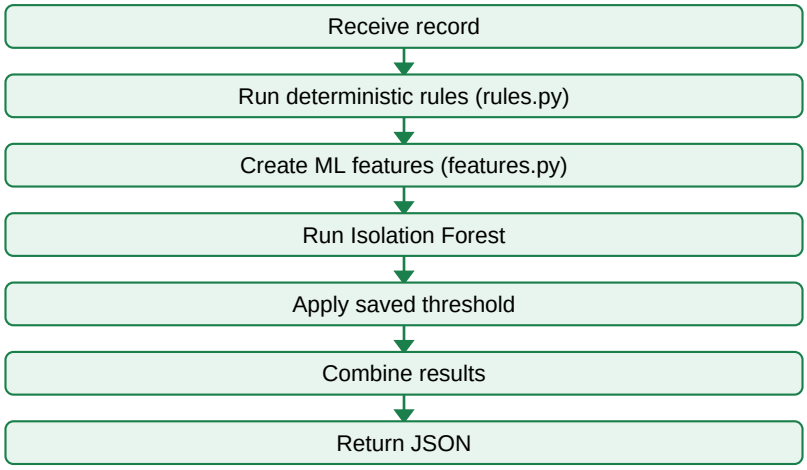
This matches the existing AWS design (private bucket, versioning on, block public access).

7. What Lambda should do

FORECAST LAMBDA



ANOMALY LAMBDA



8. Exact files Lambda needs

Lambda	Files from S3
forecast	model.onnx, preprocessor.joblib, features.py, model_metadata.json
anomaly	isolation_forest.joblib, preprocessor.joblib, threshold.json, rules_config.json, features.py, rules.py, model_metadata.json

9. Python packages Lambda needs

Lambda	Packages (exact versions)	Local install size*
forecast	onnxruntime==1.31.0, numpy==2.5.1, joblib==1.6.0	~102.9 MB
anomaly	scikit-learn==1.9.0, numpy==2.5.1, scipy==1.18.1, joblib==1.6.0	~232.4 MB

*Measured on the Windows development machine. Build Linux wheels for Lambda (manylinux, x86_64 or aarch64). Put scikit-learn/scipy/numpy in a Lambda layer. Limits: 50 MB zipped direct upload, 250 MB unzipped.

10. Sample forecast request


```
{
 "company_id": "COMP_0001",
 "product_id": "PROD_000001",
 "industry": "Beverages",
 "history": [
  {
   "month": "2025-12",
   "packaging_kg": 419.017,
   "units_sold": 9395,
   "weight_per_unit_g": 44.6
  },
  {
   "month": "2026-01",
   "packaging_kg": 406.083,
   "units_sold": 9105,
   "weight_per_unit_g": 44.6
  },
  {
   "month": "2026-02",
   "packaging_kg": 511.74,
   "units_sold": 11474,
   "weight_per_unit_g": 44.6
  }
 ]
}
(only the last 3 of 13 history months shown)
```


11. Sample forecast response


```
{
 "company_id": "COMP_0001",
 "product_id": "PROD_000001",
 "target_month": "2026-03",
 "forecast_kg": 603.748,
 "model_version": "forecast-v1",
 "model_name": "xgboost",
 "status": "success"
}
```


12. Sample anomaly request (abbreviated)


```
{
 "record_type": "invoice",
 "record_id": "INV_00017685",
 "company_id": "COMP_0001",
 "invoice": {
  "invoice_id": "INV_00017685",
  "invoice_number": "SYEC/2526/00004",
  "invoice_type": "PKG_PURCHASE",
  "company_id": "COMP_0001",
  "supplier_id": "SUP_TEST_001",
  "buyer_id": "COMP_0001",
  "invoice_date": "2025-11-17",
  "status": "issued",
  "supplier_gstin": "06ODQCM2783K1Z4",
  "buyer_gstin": "07OLWCZ2975K3ZF",
  "currency": "INR",
  "fx_rate": null,
  "total_taxable_value": 105529.09,
  "gst": 18995.24,
  "total": 124524.33,
  "original_invoice_id": null
 },
 "lines": [
  {
   "line_no": 1,
   "item_description": "PET bottle for 750 ml - PET",
   "product_id": "PROD_000001",
   "component_id": "PKGCOMP_000002",
   "material": "PET",
   "plastic_category": "CAT_I",
   "hsn_code": "3923",
   "raw_quantity": 0.5,
   "raw_unit": "MT",
   "weight_per_unit_g": null,
   "normalized_quantity_kg": 500.0,
   "recorded_quantity_kg": 500000.0,
   "rate": 211058.17,
   "taxable_value": 105529.09,
```

```
    "gst_rate": 18,
    "gst_amount": 18995.24,
    "transaction_id": "TXN_00121740",
    "transaction": {
     "transaction_id": "TXN_00121740",
     "transaction_type": "purchase",
     "status": "posted",
     "raw_quantity": 500.0,
     "raw_unit": "MT",
     "normalized_quantity_kg": 500000.0,
     "supplier_id": "SUP_TEST_001",
     "date": "2025-11-17"
    },
    "master_component": {
     "material": "PET",
     "synthetic_category": "CAT_I",
     "weight_per_unit_g": 25.0,
     "component_form": "bottle"
    }
   }
  ],
  "context": {
   "item_month_qty": {
    "1": 500000.0
   },
   "item_median_qty": {
    "1": 30.1
   },
   "item_prev_month_qty": {
    "1": 49.125
   },
   "issuer_company_median_total_inr": 7496.84,
   "issuer_company_invoice_count": 13,
   "earlier_same_key_invoice_ids": []
  },
  "master": {
   "issuer_has_gstin": true,
   "issuer_supplier": {
    "supported_materials": [
     "PET"
    ],
    "supported_component_forms": [
     "bottle",
     "jar"
    ]
   }
  }
}
```


13. Sample anomaly response


```
{
 "record_id": "INV_00017685",
 "is_anomaly": true,
 "anomaly_score": 0.599222,
 "threshold": 0.53867,
 "ml_flag": true,
 "rule_findings": [
  {
   "rule_id": "QTY_MATCH_TOL",
   "severity": "medium",
   "reason": "line1: invoice 500.000 kg vs record 500000.000 kg"
  },
  {
   "rule_id": "QTY_RATIO_POW10",
   "severity": "high",
   "reason": "record differs by a power of ten on lines [1]"
  }
 ],
 "model_version": "anomaly-v1",
 "status": "success"
}
```

14. How to test locally

```
python ml/aws_examples/test_forecast_local.py
python ml/aws_examples/test_anomaly_local.py
```

Each script loads the exported files, runs the sample request through the reference handler and checks the answer equals sample_output.json. Nothing is retrained.

15. How to upload models to S3

```
aws s3 cp ml/artifacts/forecast/v1/ s3://<model-bucket>/models/forecast/v1/ --recursive
aws s3 cp ml/artifacts/anomaly/v1/  s3://<model-bucket>/models/anomaly/v1/  --recursive
```

16. How Lambda downloads and loads the model

Set the environment variable **MODEL_BUCKET**. On a cold start the handler copies the v1 folder from S3 to **/tmp/models/<model>/v1/** and keeps the loaded model in memory for warm calls. The Lambda role needs only **s3:GetObject** on **models/***.

Measured on the local machine (not AWS)	Forecast	Anomaly
Model file size	7.101 MB	1.006 MB
Load time (fresh process)	1981.9 ms	1619.9 ms
Median call time (warm)	0.15 ms	7.53 ms
Suggested starting memory	512 MB	1024 MB

17. What goes into DynamoDB

Store	Example
company_id (partition key)	COMP_0001
timestamp (sort key)	2026-10-09T10:00:00Z#forecast
model_version	forecast-v1 / anomaly-v1
analysis result	forecast_kg, or is_anomaly + anomaly_score
findings	rule_findings list
record / product id	INV_… / PROD_…

Do not store model files in DynamoDB — they stay in S3.

18. Common errors

Error	Likely cause	Fix
Missing model file	Wrong S3 prefix or MODEL_BUCKET	Check models/<model>/v1/
Wrong feature order	Files from different versions mixed	Upload the whole v1 folder; check SHA256SUMS
Missing feature / KeyError	Request missing a field	Follow the sample request
Wrong Python dependency version	Different numpy/onnxruntime/sklearn	Use the exact pinned versions
S3 AccessDenied	Role lacks s3:GetObject	Grant read on models/* only
numpy JSON serialization error	Returning numpy values	Handlers convert to plain float/bool
Slow first call	Cold start loads libraries and model	Keep the model in a global; size memory

19. How to verify integration

1) Run both local tests. 2) Deploy each Lambda with the same sample event; compare with sample_output.json. 3) Check the result row in DynamoDB has model_version. 4) Send a bad request (empty history) and confirm status = error.

20. Final checklist

- ■ Both v1 folders uploaded to S3; SHA256SUMS checked
- ■ Layers built for the right architecture with pinned versions
- ■ MODEL_BUCKET set on both Lambdas
- ■ IAM: s3:GetObject on models/*, DynamoDB PutItem on the results table only
- ■ Local tests pass; the same sample passes in Lambda
- ■ Results stored with model_version and timestamp
- ■ UI labels results as synthetic demo and not a compliance determination

Generated from the artifacts on 2026-10-09 · dataset 0c8f6d5ad65954a9… · git b245219b18